EXPERIMENT NO. 2

CONTRAST ENHANCEMENT AND HISTOGRAM-BASED IMAGE PROCESSING

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
import shutil
import pandas as pd
from google.colab import files
from IPython.display import display, Image
print("=" * 70)
print("EXPERIMENT NO. 2")
print("Contrast Enhancement and Histogram-Based Image Processing")
print("=" * 70)
# -------------------- 2. UPLOAD IMAGE ----------------------------
print("\n📁 Select a low-contrast image from your PC...")
uploaded = files.upload()
if len(uploaded) == 0:
    raise Exception("❌ No image uploaded.")
input_filename = list(uploaded.keys())[0]
print(f"\n✅ Image uploaded: {input_filename}")
# -------------------- 3. CREATE OUTPUT FOLDER -------------------
output_dir = "/content/Experiment_02_Results"
if os.path.exists(output_dir):
    shutil.rmtree(output_dir)
os.makedirs(output_dir)
# -------------------- 4. READ IMAGE -----------------------------
original_color = cv2.imread(input_filename)
if original_color is None:
    raise Exception("❌ Could not read the uploaded image.")
# Convert color image to grayscale
gray = cv2.cvtColor(original_color, cv2.COLOR_BGR2GRAY)
```

```
    os.path.join(output_dir, "02_Contrast_Stretching.png"),
    contrast_stretched
)
cv2.imwrite(
    os.path.join(output_dir, "03_Histogram_Equalization.png"),
    hist_equalized
)
cv2.imwrite(
    os.path.join(output_dir, "04_CLAHE.png"),
    clahe_image
)
# -------------------- 10. ORIGINAL IMAGE ----------------------
plt.figure(figsize=(10, 7))
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image", fontsize=16)
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(output_dir, "05_Original_Image_Display.png"),
    dpi=200,
    bbox_inches="tight"
)
plt.show()
# -------------------- 11. ORIGINAL HISTOGRAM -------------------
plt.figure(figsize=(10, 6))
plt.hist(
    gray.ravel(),
    bins=256,
    range=(0, 256)
)
plt.title("Histogram of Original Image", fontsize=16)
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
```

```
plt.title(
    "Histogram Equalization",
    fontsize=16
)
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(output_dir, "08_Histogram_Equalization_Display.png"),
    dpi=200,
    bbox_inches="tight"
)
plt.show()
# -------------------- 14. CLAHE DISPLAY ------------------------
plt.figure(figsize=(10, 7))
plt.imshow(
    clahe_image,
    cmap="gray"
)
plt.title(
    "CLAHE Enhanced Image",
    fontsize=16
)
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(output_dir, "09_CLAHE_Display.png"),
    dpi=200,
    bbox_inches="tight"
)
plt.show()
```

```
plt.show()
# -------------------- 16. IMAGE COMPARISON ---------------------
fig, axes = plt.subplots(2,2, figsize=(14, 11)
)
comparison_images = [
    (gray, "Original Image"),
    (contrast_stretched, "Contrast Stretching"),
    (hist_equalized, "Histogram Equalization"),
    (clahe_image, "CLAHE")
]
for ax, (img, title) in zip(
    axes.ravel(),
    comparison_images
):
    ax.imshow(
        img,
        cmap="gray"
    )
 ax.set_title(
        title,
        fontsize=16
    )
    ax.axis("off")
plt.suptitle(
    "Contrast Enhancement Techniques Comparison",
    fontsize=20
)
plt.tight_layout()
plt.savefig(
    os.path.join(output_dir, "11_Final_Image_Comparison.png"),
    dpi=250,
    bbox_inches="tight"
)
```

```
        )
        axes[i, 1].set_ylabel(
            "Frequency"
        )
        axes[i, 1].set_xlim([0, 256])
        axes[i, 1].grid(alpha=0.25)
    plt.suptitle(
        "Complete Contrast Enhancement Analysis",
        fontsize=22
    )
    plt.tight_layout()
    plt.savefig(
        os.path.join(output_dir, "12_Complete_Analysis.png"),
        dpi=200,
        bbox_inches="tight"
    )
    plt.show()
    # -------------------- 18. STATISTICAL ANALYSIS ----------------
    def calculate_statistics(img):
        return {
            "Mean Intensity": np.mean(img),
            "Standard Deviation": np.std(img),
            "Minimum Intensity": np.min(img),
            "Maximum Intensity": np.max(img),
            "Dynamic Range": int(
                np.max(img) - np.min(img)
            )
        }
statistics = {
    "Original":
        calculate_statistics(gray),
    "Contrast Stretching":
        calculate_statistics(contrast_stretched),
```

```
6. Histograms of the enhanced images were compared.

7. Statistical parameters such as mean intensity and standard deviation
were calculated.

8. The processed images demonstrate the effect of different

   histogram-based contrast enhancement techniques.

""")

# -------------------- 23. LIST ALL GENERATED RESULTS ----------

print("\n" + "=" * 70)

print("GENERATED RESULT FILES")

print("=" * 70)

for filename in sorted(os.listdir(output_dir)):

print("✓", filename)

print("\n" + "=" * 70)

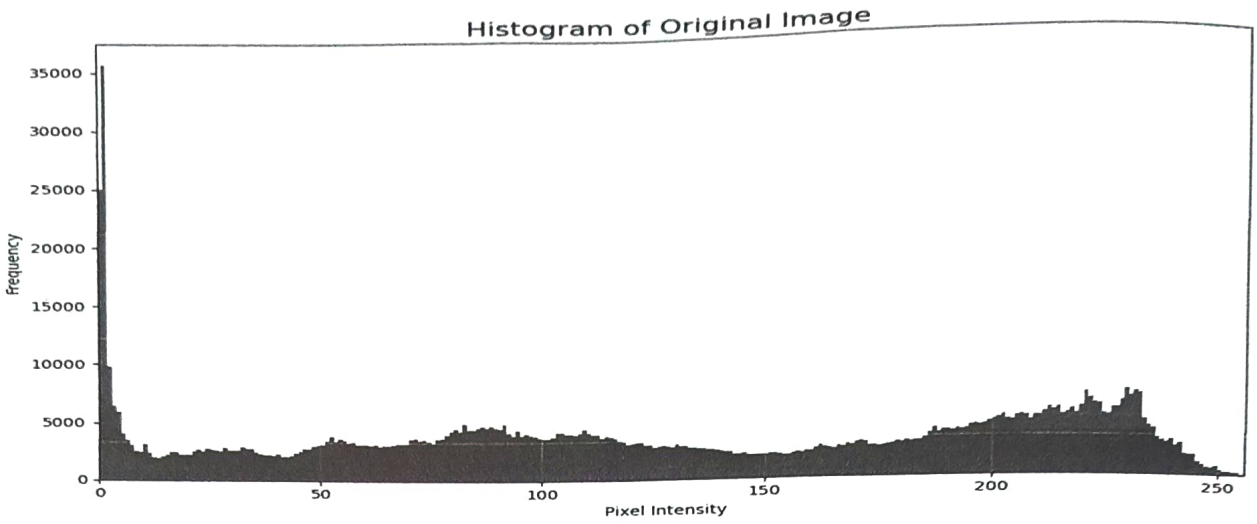
print("☑ EXPERIMENT NO. 2 COMPLETED SUCCESSFULLY")

print("=" * 70)

print(

    f"\nAll result files are available inside:\n{output_dir}" )
```



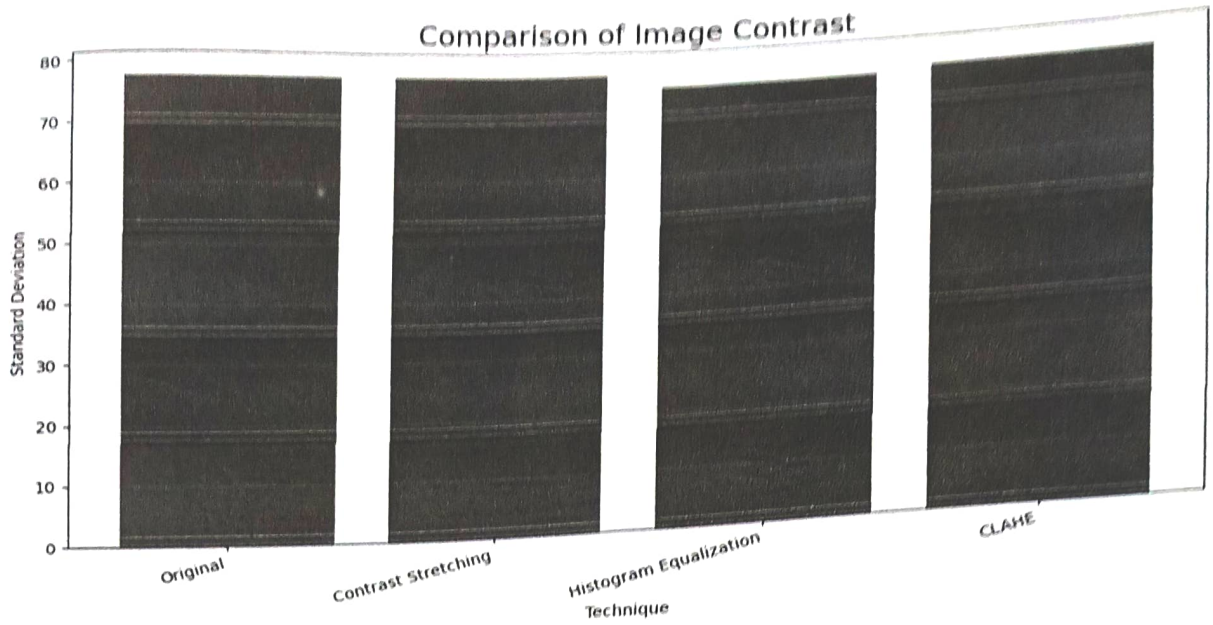


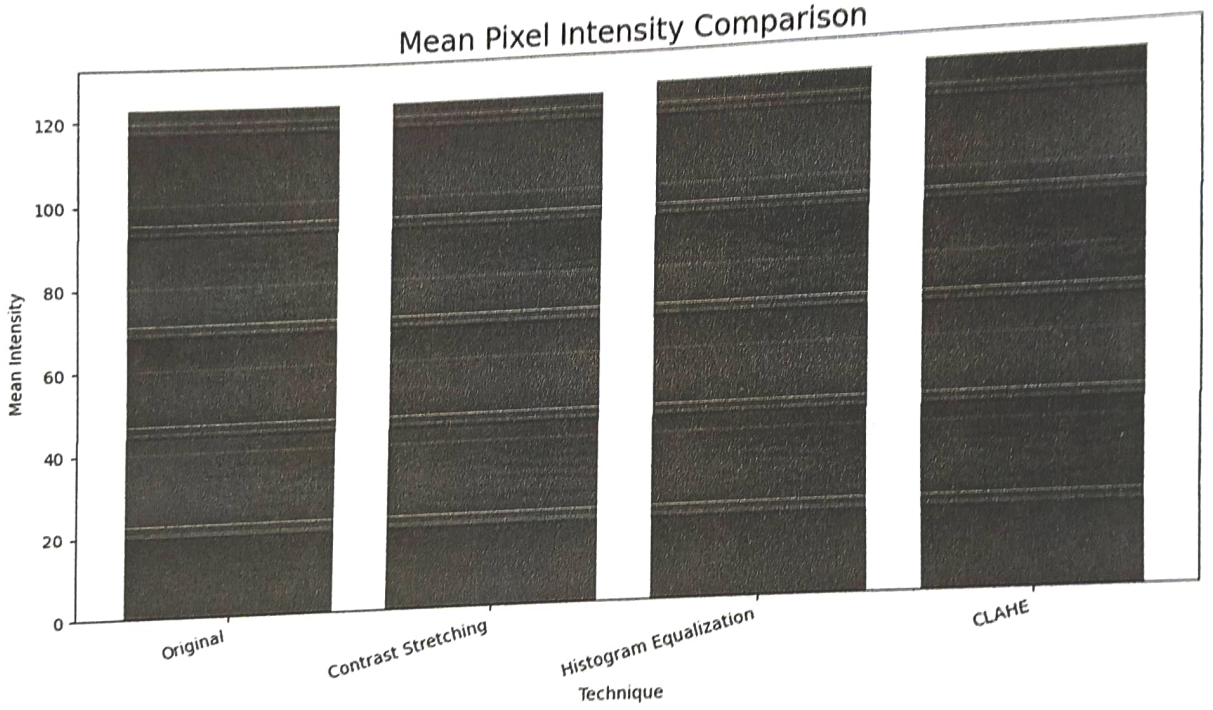
Contrast Stretching



Histogram Equalization



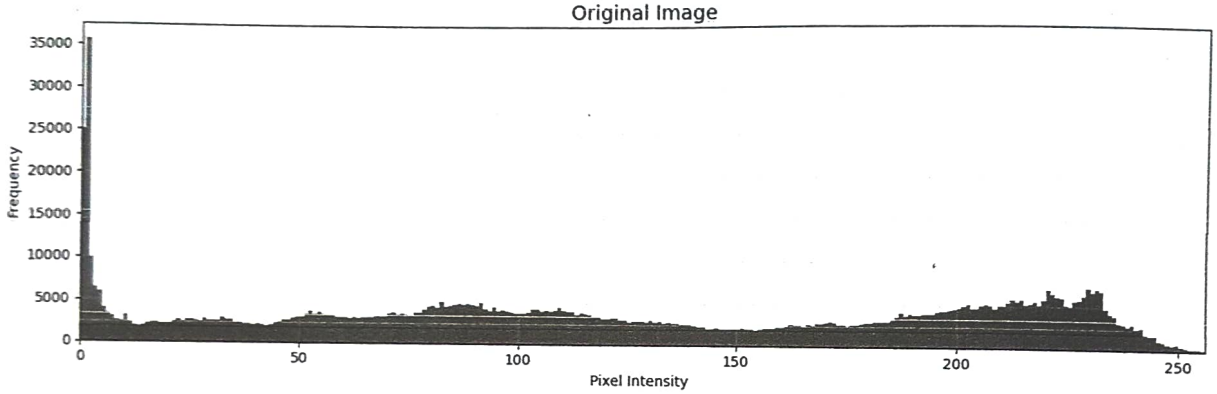

Comparison of Image Contrast
Standard Deviation
0
10
20
30
40
50
60
70
80
Original
Contrast Stretching
Histogram Equalization
CLAHE
Technique


Mean Pixel Intensity Comparison
Mean Intensity
0
20
40
60
80
100
120
Original
Contrast Stretching
Histogram Equalization
CLAHE
Technique

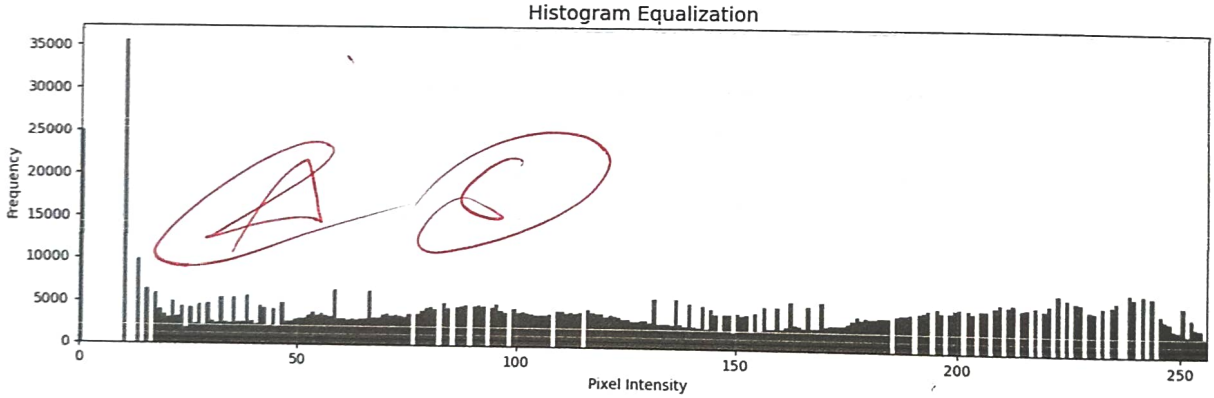
CLAHE Enhanced Image



Original Image



Histogram Equalization



CLAHE

